

Chap 6~7

lecture notes

Date

No.

6 Types and Conversions

type definitions

typedef is used to create aliases (alternative names) for existing data types. It does not create a new type, but simply allows you to use a new name for an existing type, which can make code more readable & easier to maintain.

Syntax: `typedef existing_type new_name;`

eg. `typedef unsigned int uint;`

`uint age=25; // Same as: unsigned int age=25;`

here, ~~uint~~ `uint` is now an alias for `unsigned int`.

eg. (Struct alias)

`typedef struct {`

`int x;`

`int y;`

`} Point;`

`Point p1={10, 20}; // instead of: struct { int x; int y; } p1;`

type `size_t` and `sizeof`

1. `size_t`

`<stdlib.h> <stddef.h>`

- `size_t` is an unsigned integer type defined in the C standard lib.
- its primary purpose is to represent sizes of objects (in bytes) or array ^{indices}.

eg. `#include <stdio.h>      size_t bytes = sizeof(arr);      // size in bytes`
`#include <stddef.h>      printf("Array size: %zu bytes\n", bytes);`
`int main () {      return 0;`
`int arr[10];      }`
`size_t n=10;`

`n` stores the # of elements

- `size_t` is a platform-independent type specifically used to represent sizes, ensuring it's large enough to represent the size of any object.
- `unsigned int` is a fixed-size unsigned integer, which does not guarantee it can represent the size of all objects (especially on 64-bit systems)

2. `sizeof`

- `sizeof` is a compile-time operator that returns the size in bytes of a type or expression
- can be applied to a type or a variable
- return type is always `size_t`

eg. `sizeof(char)` // usually 1
 `int` 4
 `double` 8

eg. Arrays

- When apply to an array, `sizeof` returns the total # of bytes occupied by the array, not the # of elements

`int arr[10];`

`sizeof(arr)` // $10 * \text{sizeof(int)} = 40$ bytes (if `int=4`) The size of the array
`sizeof(arr) / sizeof(arr[0])` // 10 the # of elements

- exception • When `sizeof` is applied to an array variable itself, it does not decay.
- The compiler knows the exact type and length of the array, so `sizeof(arr)` returns total bytes of the array (# of elements $\times$ size of each element)

⚠ if you pass an array to a function, it decays to a pointer, and `sizeof` on the parameter gives the size of the pointer, not the whole array itself.
 the # of bytes needed to store the pointer

32-bit sys : pointer size = 4 bytes 64 : 8 bytes.

type cast

see tutorial 2

Enumerated type

- The constants defined in an enum are called enumeration constants
 an enum specifies an integer type and defines a set of named constants to represent the possible values of that type
- like character constants, they have type `int`.
- By default, the first constant is 0, and the value increases by 1 for each subsequent constant, unless explicitly assigned.

`enum workday { MON=1, TUE, WED, THUR, FRI };`

`int day;`

`for (day = MON; day <= FRI; day++)`

`printf("%d", day);` // 1 2 3 4 5

output :

format strings for scanf()

Conversion specification

- In `scanf()`, a conversion specification defines the format of the value to be read
- the corresponding argument must be the address of the variable where the value will be stored.

`int a;`

`scanf("%d", &a);` // %d reads an int, &a is the address to store the value

literal character mismatch

- if a literal character (other than a space) in the format does not match the corresponding input character, `scanf()` stops immediately
- No further input is read, and no further values are stored.

eg. `double x;`

`scanf("***$%lf", &x);`

`***$ 25.99` → okay, `x = 25.99`

assignment suppression

- using `*` in the conversion specification means read but not store the value

`**$ 25.99` → `x`. `scanf` stops before reading the num.

eg. `double x;`

`scanf("%*d%lf", &x);` // skip an int, then reads a double into `x`

• input: `50 2.99`

• result: `x = 2.99`

7 macros

Syntax

1. Always put `( )` around each parameter inside the macro body
2. - - - - - the whole expression, otherwise ↲

`Square(x) x*x`

`square(2+3) → 2+3*2+3`